PID算法简介

- PID是非常经典的控制算法，因其简单使用，并且鲁棒性强而经久不衰
- PID是一种闭环控制算法，它动态改变施加到被控对象的输出值（Out），使 得被控对象某一物理量的实际值（Actual），能够快速、准确、稳定地跟踪到 指定的目标值（Target）
- PID是一种基于误差（Error）调控的算法，其中规定：误差=目标值-实际值， PID的任务是使误差始终为0
- PID对被控对象模型要求低，无需建模，即使被控对象内部运作规律不明确， PID也能进行调控

$$out(t) = Kp * error(t) + Ki * \int [0,t] error(t)dt + Kd * d(error(t))/dt$$

比例项

P控制器：

$$out(t) = Kp * error(t)$$

- 比例项的输出值仅取决于当前时刻的误差，与历史时刻无关。当前存在误差 时，比例项输出一个与误差呈正比的值，当前不存在误差时，比例项输出0
- Kp越大，比例项权重越大，系统响应越快，但超调也会随之增加

积分项

PI控制器

$$out(t) = Kp * error(t) + Ki * \int [0-t] error(t)dt$$

- 积分项的输出值取决于0~t所有时刻误差的积分，与历史时刻有关。积分项将 历史所有时刻的误差累积，乘上积分项系数Ki后作为积分项输出值
- 积分项用于弥补纯比例项产生的稳态误差，若系统持续产生误差，则积分项 会不断累积误差，直到控制器产生动作，让稳态误差消失
- Ki越大，积分项权重越大，稳态误差消失越快，但系统滞后性也会随之增加

微分项

PID控制器

$$out(t) = Kp * error(t) + Ki * \int [0,t] error(t)dt + Kd * d(error(t))/dt$$

- 微分项的输出值取决于当前时刻误差变化的斜率，与当前时刻附近误差变化 的趋势有关。当误差急剧变化时，微分项会负反馈输出相反的作用力，阻碍 误差急剧变化
- 微分项给系统增加阻尼，可以有效防止系统超调，尤其是惯性比较大的系统
- Kd越大，微分项权重越大，系统阻尼越大，但系统卡顿现象也会随之增加

连续形式离散化

$$out(k) = Kp * error(k) + Ki * T\Sigma[j=0,k] error(j) + Kd * (error(k) - error(k-1))/T$$

位置式PID算法

```
/*获取实际值*/
Actual = 读取传感器();

/*获取本次误差和上次误差*/
Error1 = Error0;            //获取上次误差
Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值
```

```
/*误差积分（累加）*/
/*如果Ki不为0，才进行误差积分，这样做的目的是便于调试*/
/*因为在调试时，我们可能先把Ki设置为0，这时积分项无作用，误差消除不了，误差积分会积累到很大的值*/
/*后续一旦Ki不为0，那么因为误差积分已经积累到很大的值了，这就导致积分项疯狂输出，不利于调试*/
if (Ki != 0)                //如果Ki不为0
{
    ErrorInt += Error0;     //进行误差积分
}
else                        //否则
{
    ErrorInt = 0;           //误差积分直接归0
}

/*PID计算*/
/*使用位置式PID公式，计算得到输出值*/
Out = Kp * Error0 + Ki * ErrorInt + Kd * (Error0 - Error1);

/*输出限幅*/
if (Out > 100) {Out = 100;}     //限制输出值最大为100
if (Out < -100) {Out = -100;}   //限制输出值最小为100

/*执行控制*/
/*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
输出至被控对象(out);
```

位置式PID由连续形式PID直接离散得到，每次计算得到的是全量的输出值， 可以直接给被控对象。

$$out(k) = Kp * error(k) + Ki * \Sigma[j = 0k]error(j) * Kd + (error(k) - error(k-1))$$

增量式PID算法

```
/*获取实际值*/
Actual = 读取传感器();

/*获取本次误差、上次误差和上上次误差*/
Error2 = Error1;            //获取上上次误差
Error1 = Error0;            //获取上次误差
Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值

/*PID计算*/
/*使用增量式PID公式，计算得到输出值*/
Out += Kp * (Error0 - Error1) + Ki * Error0
        + Kd * (Error0 - 2 * Error1 + Error2);

/*输出限幅*/
if (Out > 100) {Out = 100;}     //限制输出值最大为100
if (Out < -100) {Out = -100;}   //限制输出值最小为100

/*执行控制*/
/*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
输出至被控对象(out);
```

增量式PID由位置式PID当前时刻和上一时刻两式相减得到

$$out(k) = Kp * error(k) + Ki * \Sigma(j = 0k)error(j) + Kd * (error(k) - error(k-1))$$

当$k = k - 1$时：$out(k-1) = Kp * error(k-1) + Ki * \Sigma(j = 0k-1)error(j) + Kd * (error(k-1) - error(k-2))$

两式相减$\Delta out(k) = Kp * (error(k) - error(k-1)) + Ki * error(k) + Kd * (error(k) - 2error(k-1) + error(k-2))$

增量式PID由位置式PID推导得到，每次计算得到的是输出值的增量，如果直 接给被控对象，则需要被控对象内部有积分功能

PID算法的改进

积分限幅

- 要解决的问题：如果执行器因为卡住、断电、损坏等原因不能消除误差，则 误差积分会无限制加大，进而达到深度饱和状态，此时PID控制器会持续输出 最大的调控力，即使后续执行器恢复正常，PID控制器在短时间内也会维持最 大的调控力，直到误差积分从深度饱和状态退出
- 积分限幅实现思路：对误差积分或积分项输出进行判断，如果幅值超过指定 阈值，则进行限制

```
/*获取实际值*/
Actual = 读取传感器();

/*获取本次误差和上次误差*/
Error1 = Error0;            //获取上次误差
Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值

/*误差积分（累加）*/
ErrorInt += Error0;

/*积分限幅*/
if (ErrorInt > 500) {ErrorInt = 500;}       //限制误差积分最大为500
if (ErrorInt < -500) {ErrorInt = -500;}     //限制误差积分最小为-500

/*
    另一种实现思路：对整个积分项进行限幅
    ErrorInt += Ki * Error0;
    if(ErrorInt > 500) {ErrorInt = 500;}
    if (ErrorInt < -500) {ErrorInt = -500;}
*/

/*PID计算*/
/*使用位置式PID公式，计算得到输出值*/
Out = Kp * Error0 + Ki * ErrorInt + Kd * (Error0 - Error1);

/*输出限幅*/
if (Out > 100) {Out = 100;}     //限制输出值最大为100
if (Out < -100) {Out = -100;}   //限制输出值最小为100

/*执行控制*/
输出至被控对象(out);
```

积分分离

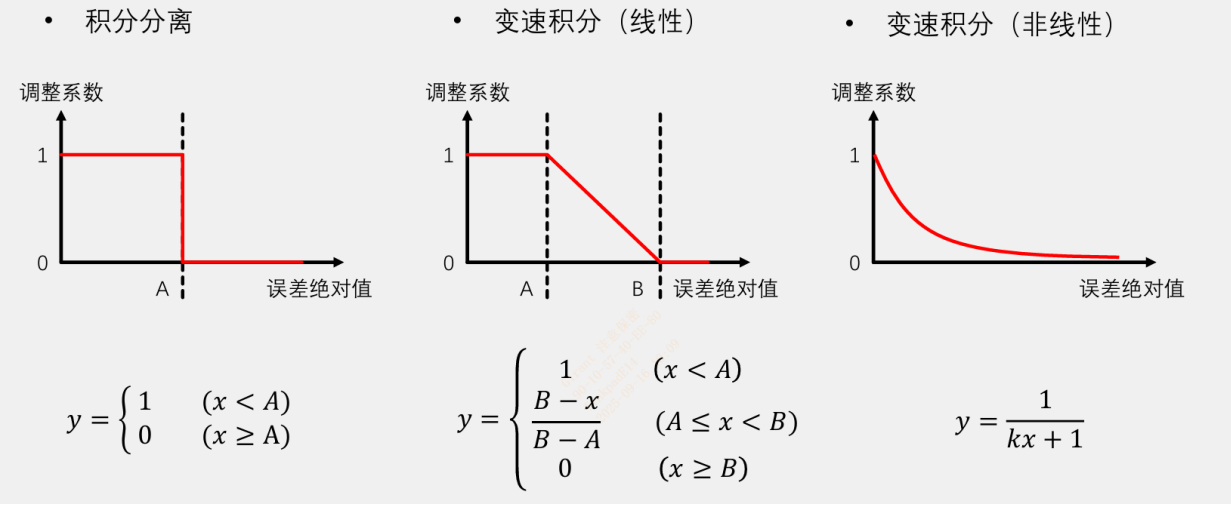
- 要解决的问题：积分项作用一般位于调控后期，用来消除持续的误差，调控前期一般误差较大且不需要积分项作用，如果此时仍然进行积分，则调控进行到后期时，积分项可能已经累积了过大的调控力，这会导致超调
- 积分分离实现思路：对误差大小进行判断，如果误差绝对值小于指定阈值， 则加入积分项作用，反之，则直接将误差积分清零或不加入积分项作用

```
/*获取实际位置值*/
Actual += Encoder_Get();    //例子是对电机进行定位置控制，周期内对速度进行累加得到位移

/*获取本次误差和上次误差*/
Error1 = Error0;            //获取上次误差
Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值

/*误差积分+积分分离*/
if (fabs(Error0) < 50)      //如果当前误差值小于指定的阈值
{
    ErrorInt += Error0;     //才进行正常的误差积分
}
else                        //否则，即当前误差值过大
{
```

```
        ErrorInt = 0;            //此时不进行积分，同时把误差积分直接清零
    }

    /*PID计算*/
    /*使用位置式PID公式，计算得到输出值*/
    Out = Kp * Error0 + Ki * ErrorInt + Kd * (Error0 - Error1);

    /*输出限幅*/
    if (Out > 100) {Out = 100;}      //限制输出值最大为100
    if (Out < -100) {Out = -100;}    //限制输出值最小为100

    /*执行控制*/
    /*输出值给到电机PWM*/
    /*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
    输出至被控对象(out);
```

变速积分

- 要解决的问题：如果积分分离阈值没有设定好，被控对象正好在阈值之外停 下来，则此时控制器完全没有积分作用，误差不能消除
- 变速积分实现思路：变速积分是积分分离的升级版，变速积分需要设计一个 函数值随误差绝对值增大而减小的函数，函数值作为调整系数，用于调整误 差积分的速度或积分项作用的强度




```
    /*获取实际位置值*/
    Actual += Encoder_Get();     //例子是对电机进行定位置控制，周期内对速度进行累加得到位移

    /*获取本次误差和上次误差*/
    Error1 = Error0;             //获取上次误差
    Error0 = Target - Actual;    //获取本次误差，目标值减实际值，即为误差值

    /*定义一个系数C，表示积分的速度，C的值与误差绝对值大小呈反比，误差绝对值越大，积分速度越慢*/
    float C = 1 / (0.2 * fabs(Error0) + 1);      //根据公式计算得到系数C

    /*误差积分*/
    ErrorInt += C * Error0;      //积分的速度由C确定，C的取值范围是0~1

    /*注意：变速积分本身没有防止积分饱和的效果，为了避免积分饱和，此处可以再加入积分限幅的逻辑*/

    /*PID计算*/
    /*使用位置式PID公式，计算得到输出值*/
    Out = Kp * Error0 + Ki * ErrorInt + Kd * (Error0 - Error1);

    /*输出限幅*/
    if (Out > 100) {Out = 100;}      //限制输出值最大为100
    if (Out < -100) {Out = -100;}    //限制输出值最小为100

    /*执行控制*/
```

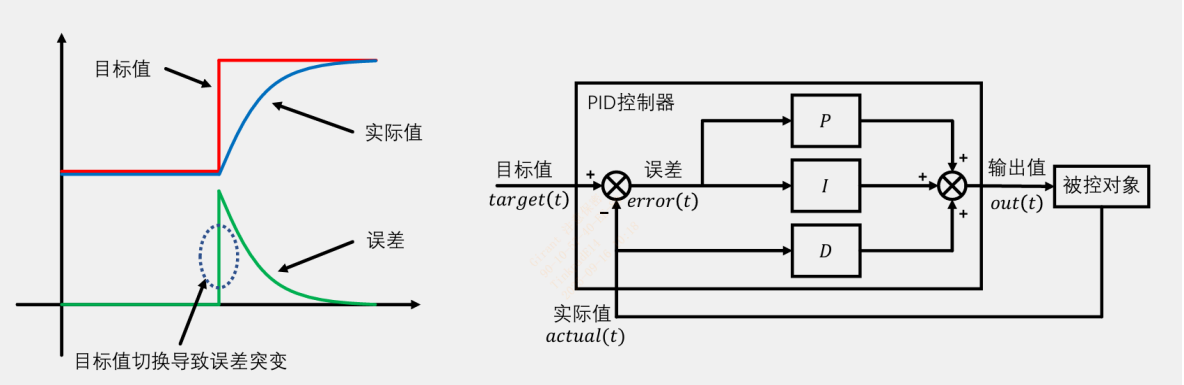
```
        /*输出值给到电机PWM*/
        /*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
        输出至被控对象(out);
```

微分先行

- 要解决的问题：普通PID的微分项对误差进行微分，当目标值大幅度跳变时， 误差也会瞬间大幅度跳变，这会导致微分项突然输出一个很大的调控力，如 果系统的目标值频繁大幅度切换，则此时的微分项不利于系统稳定,但是加入微分先行会让系统相应出现迟滞
- 微分先行实现思路：将对误差的微分替换为对实际值的微分

$$\text{普通}PID\text{的微分项输出：}dout(k) = Kd * (error(k) - error(k-1))$$

$$\text{微分先行}PID\text{的微分项输出：}dout(k) = -Kd * (actual(k) - actual(k-1))$$




```
        /*获取实际位置值*/
        //例子是对电机进行定位置控制，周期内对速度进行累加得到位移
        Actual1 = Actual;           //获取上次实际值
        Actual += Encoder_Get();    //获取本次实际值

        /*获取本次误差和上次误差*/
        Error1 = Error0;            //获取上次误差
        Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值

        /*误差积分（累加）*/
        /*如果Ki不为0，才进行误差积分，这样做的目的是便于调试*/
        /*因为在调试时，我们可能先把Ki设置为0，这时积分项无作用，误差消除不了，误差积分会积累到很大的值*/
        /*后续一旦Ki不为0，那么因为误差积分已经积累到很大的值了，这就导致积分项疯狂输出，不利于调试*/
        if (Ki != 0)                //如果Ki不为0
        {
            ErrorInt += Error0;     //进行误差积分
        }
        else                        //否则
        {
            ErrorInt = 0;           //误差积分直接归0
        }

        /*PID计算，先得到微分项*/
//      DifOut = Kd * (Error0 - Error1);        //这一句是普通PID的微分项计算公式
        DifOut = - Kd * (Actual - Actual1);     //这一句是微分先行的微分项计算公式
                                                //计算结果要取负，因为实际值的变化趋势和误差变化趋势相反

        /*PID计算*/
        /*使用位置式PID公式，计算得到输出值*/
        Out = Kp * Error0 + Ki * ErrorInt + DifOut;

        /*输出限幅*/
        if (Out > 100) {Out = 100;}     //限制输出值最大为100
        if (Out < -100) {Out = -100;}   //限制输出值最小为100

        /*执行控制*/
        /*输出值给到电机PWM*/
```

```
            /*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
            输出至被控对象(out);
```

不完全微分

- 要解决的问题：传感器获取的实际值经常会受到噪声干扰，而PID控制器中的 微分项对噪声最为敏感，这些噪声干扰可能会导致微分项输出抖动，进而影 响系统性能
- 不完全微分实现思路：给微分项加入一阶惯性单元（低通滤波器）

$$\text{普通}PID\text{的微分项输出：}dout(k) = Kd * (error(k) - error(k-1))$$

$$\text{不完全微分}PID\text{的微分项输出：}dout(k) = (1-a) * Kd * (error(k) - error(k-1)) + a * dout(k-1)$$

```
            /*获取实际位置值*/
            Actual += Encoder_Get();    //例子是对电机进行定位置控制，周期内对速度进行累加得到位移

            /*模拟给实际值添加一些噪声干扰*/
            Actual += rand() % 41 - 20;     //使用随机数生成函数rand生成随机数，并将结果限定在-20~20的范围内

            /*获取本次误差和上次误差*/
            Error1 = Error0;            //获取上次误差
            Error0 = Target - Actual;   //获取本次误差，目标值减实际值，即为误差值

            /*误差积分（累加）*/
            /*如果Ki不为0，才进行误差积分，这样做的目的是便于调试*/
            /*因为在调试时，我们可能先把Ki设置为0，这时积分项无作用，误差消除不了，误差积分会积累到很大的值*/
            /*后续一旦Ki不为0，那么因为误差积分已经积累到很大的值了，这就导致积分项疯狂输出，不利于调试*/
            if (Ki != 0)                //如果Ki不为0
            {
                ErrorInt += Error0;     //进行误差积分
            }
            else                        //否则
            {
                ErrorInt = 0;           //误差积分直接归0
            }

            /*PID计算，先得到微分项*/
//          DifOut = Kd * (Error0 - Error1);         //这一句是普通PID的微分项计算公式

            /*定义一个滤波强度系数a，a的取值范围是0~1，值越大表示滤波作用越强*/
            float a = 0.9;
            DifOut = (1 - a) * Kd * (Error0 - Error1) + a * DifOut;     //不完全微分的微分项计算公式

            /*PID计算*/
            /*使用位置式PID公式，计算得到输出值*/
            Out = Kp * Error0 + Ki * ErrorInt + DifOut;

            /*输出限幅*/
            if (Out > 100) {Out = 100;}     //限制输出值最大为100
            if (Out < -100) {Out = -100;}   //限制输出值最小为100

            /*执行控制*/
            /*输出值给到电机PWM*/
            /*因为此函数的输入范围是-100~100，所以上面输出限幅，需要给Out值限定在-100~100*/
            输出至被控对象(out);
```